Blender and Python

À. Vinacua
with some ideas from Carlos Andújar

Thursday 6th March, 2025

© Universitat Politècnica de Catalunya·BarcelonaTech, 2025. All rights reserved.

Learning Python and Blender-Python

This slides apply to versions of blender after 2.9x, using Python 3.y

Blender-python

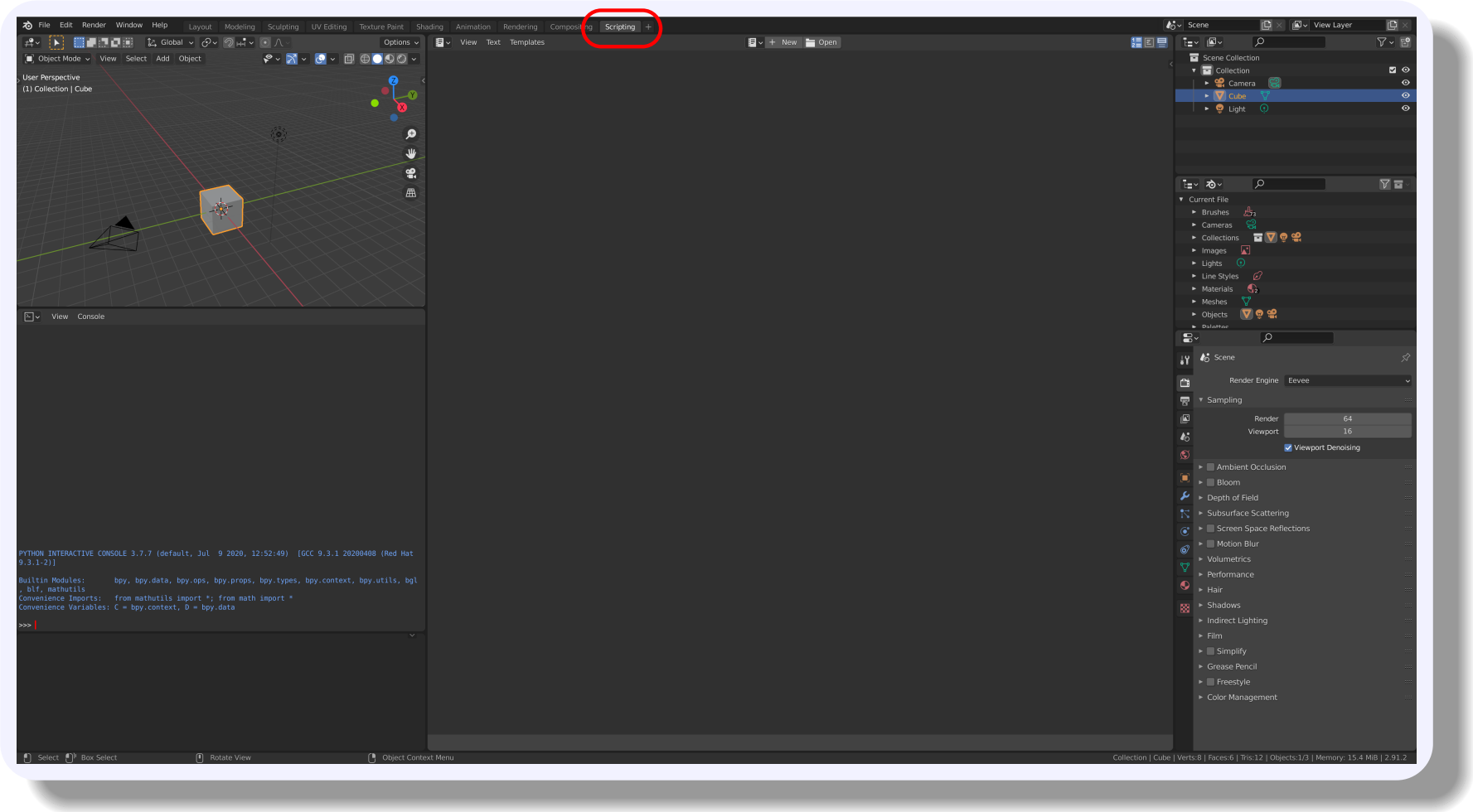
The authoritative reference for the API is
`https://docs.blender.org/api/current`

But there are many tutorials and documents on the web.
`https://en.wikibooks.org/wiki/Blender_3D:_Noob_to_Pro/Advanced_Tutorials/Python_Scripting/Introduction`
may come in handy as well (but documents version 2.67).

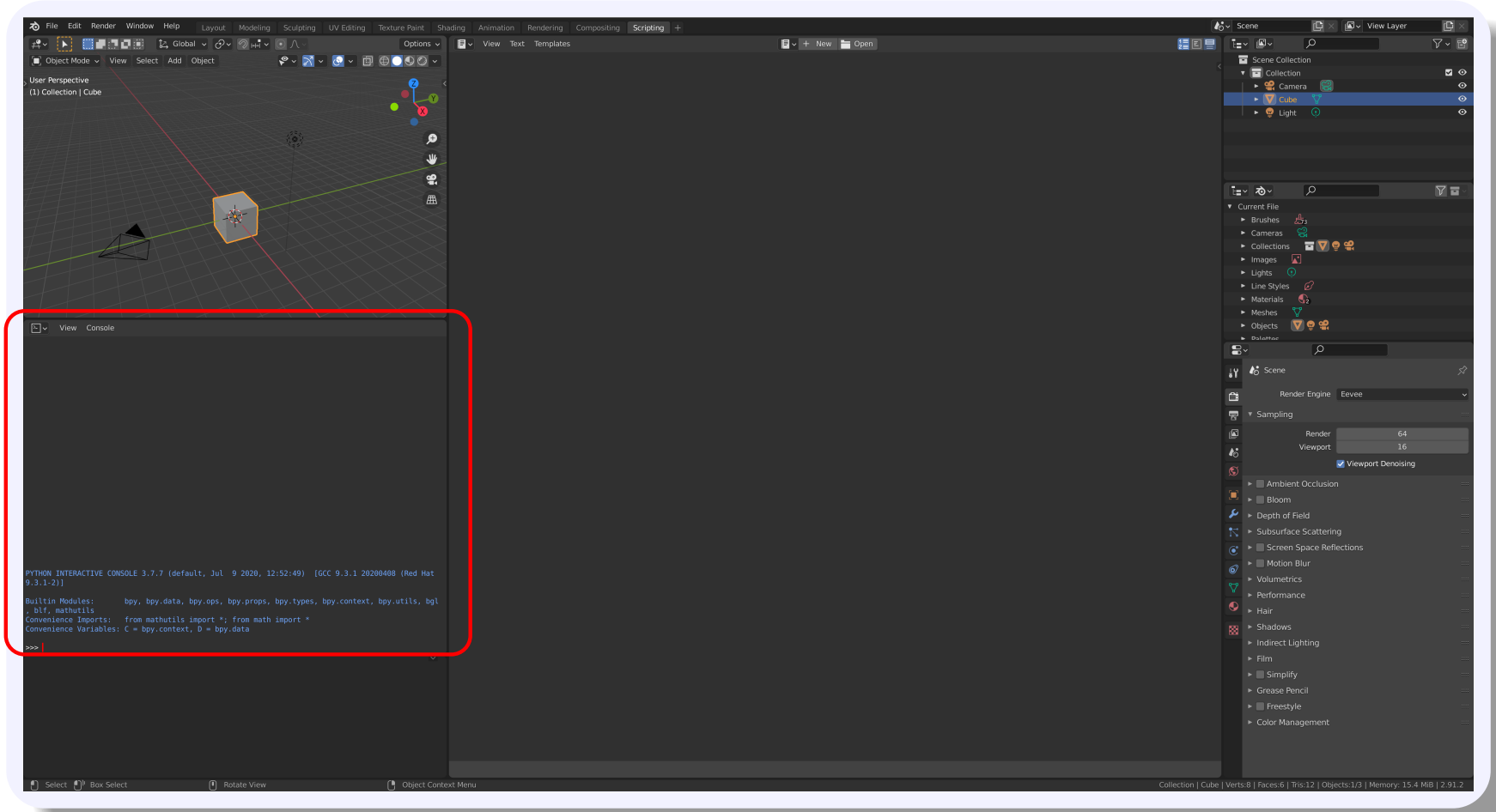
There is a blender YouTube channel (`https://www.youtube.com/channel/UCSMOQeBJ2RAnuFungnQOxLg`) that features lots of demos and tutorials...

Scripting layout

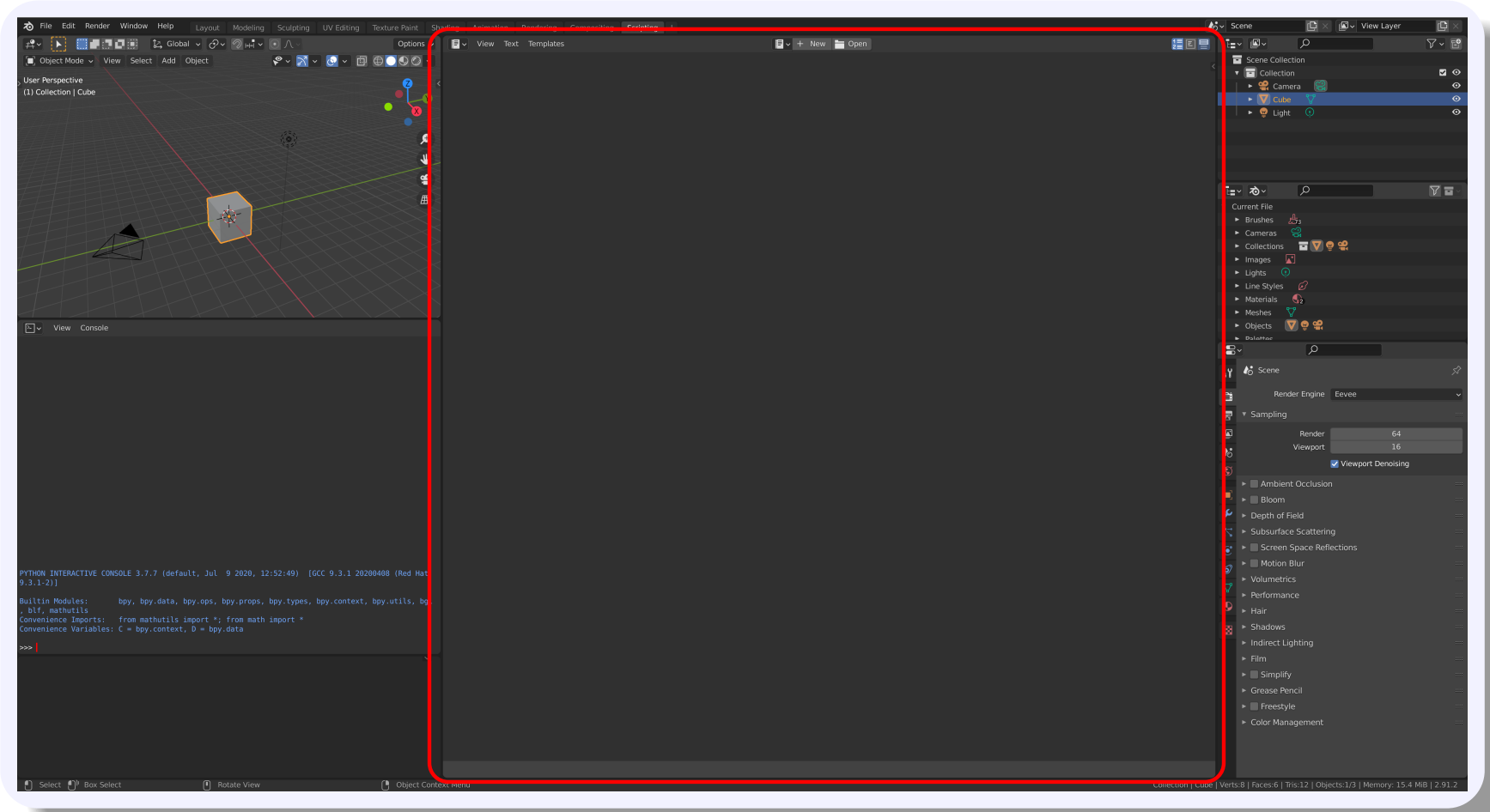
Access through the “Scripting” tab at the top



Scripting layout



Scripting layout



Blender's Python API

It is an intricate, long, and often redundant API.

Main components

```
bpy.context -> object(s) selected, mode,...
bpy.data    -> the current file
bpy.utils   -> :-)
bpy.ops     -> all defined operators
bpy.types
...
mathutils   -> things like Vector, Matrix,
               Quaternion, ...
```

Experimentation

The console within blender already has the main packages imported. You can also use `C` for `bpy.context` and `D` for `bpy.data`. In your scripts, however, you need to import whatever you need.
The console allows interactive exploration and experimentation. Furthermore, it auto-completes pressing 'Tab'.
Try entering, for example

```
dir(bpy.types.Mesh)
help(bpy.types.Mesh.from_pydata)
type(bpy.data.objects['Cube'].data)
mesh = bpy.data.objects['Cube'].data
type(mesh.vertices[2])
mesh.vertices[3].co[2]+=5.
len(mesh.polygons)
```

Working with the editor

Beware that scripts access blender's internal data, and may corrupt it and cause blender to crash! Always save your work before testing an edit. Also, there are alternatives. Enter this in blender's editor:

```
import imp
import sys
if not '' in sys.path:
    sys.path.append('')
import myscript
imp.reload(myscript)
myscript.main(<params>)
```

Or use the script `edit_external_for_blender` to edit with emacs, and do not ever touch the editor's text. You can get this from `git://github.com/ldo/edit_external_for_blender.git` or from `/assig/sgi/Sessions/lab2/edit_external.py`

More details on API

Before we go on, always keep in mind that while executing your script, blender is locked waiting and will not respond.

`bpy.data` gives you access to everything in the blender file:

```
>>> bpy.data.scenes
<bpy_collection[1], BlendDataScenes>

>>> bpy.data.objects
<bpy_collection[2], BlendDataObjects>

>>> bpy.data.objects.keys()
['Cube', 'Torus']

>>> bpy.data.objects[:]
[bpy.data.objects['Cube'], bpy.data.objects['Toru
```

These collections include methods to create and destroy data objects:

```
>>> mymesh = D.meshes.new("newMesh")
>>> mymesh
bpy.data.meshes['newMesh']

>>> bpy.data.meshes.remove(mymesh)
>>>
```

Care must be taken to link all these pieces properly. We will return to this in the next session.

`bpy.data` accesses the whole information in the blender file. More often, we want our scripts to acces information depending on the actions of the user or the state of the app.
Enter the `bpy.context` core module. It allows simple access to the selected object(s), active tools...

```
>>> bpy.context.active_object
bpy.data.objects['Cube']
>>> bpy.context.selected_objects
[bpy.data.objects['Cube'],
  bpy.data.objects['Cube.001'],
  bpy.data.objects['Cube.002'], ...
```

bpy.ops

This module contains all operators.

These can be attached to buttons, menu entries, or (sometimes) called from your code.

You can add your own, but need to abide by rules. See `https://docs.blender.org/api/current/info_quickstart.html#operators-tools`

A complete list (there are over 2000!) can be extracted via 'Help⟶Operator Cheat Sheet' (if Developer extras has been checked)

Meshes

Not to be confused with BMesh! It accesses data in object mode, and is designed to be succinct.

Mesh stores 4 main arrays:

- `Mesh.vertices` array of triples of floats
- `Mesh.edges` array of pairs of positive integers
- `Mesh.loops` array of **corners** of the mesh grouped by polygon
- `Mesh.polygons` array of faces of the mesh

Each of them carries a lot of other information. For example, vertices store also `normals`. Notice that `loops` indexes corners, so its indices may be used elsewhere for things like texture coordinates.

Example: info_mesh

```
import bpy
from time import time
```

Example: info_mesh

```
import bpy
from time import time
...
def main():
    ob = bpy.context.active_object
    if not ob or ob.type != 'MESH':
        print("Active object is not a MESH! Abort
        return

    bpy.ops.object.mode_set(mode='OBJECT')
    mesh = ob.data

    t = time()
    processa_malla(mesh)
    print("Script took %6.2f secs.\n\n"
          % (time()-t))
```

```python
def processa_malla(me):
    print('Name of the mesh: %s' % (me.name))
    print(' V= %d' % (len(me.vertices)))
    print(' E= %d' % (len(me.edges)))
    print(' F= %d' % (len(me.polygons)))
```

```python
def processa_malla(me):
    print('Name of the mesh: %s' % (me.name))
    print(' V= %d' % (len(me.vertices)))
    print(' E= %d' % (len(me.edges)))
    print(' F= %d' % (len(me.polygons)))

print('Vertex list:')
for i in range(len(me.vertices)):
    coord=me.vertices[i].co
    print(i, ":", coord[0], coord[1], coord[2])
    # coord.x, coord.y,... would do the same
```

```
print('Face list:')
for i in range(len(me.polygons)):
    print(i, ":", me.polygons[i].vertices[:])
```

```
print('Face list:')
for i in range(len(me.polygons)):
    print(i, ":", me.polygons[i].vertices[:])
print('Equiv_________________')
for poly in me.polygons:
    print("Polygon index: %d, length: %d"
          % (poly.index, poly.loop_total))
    for loop_index in poly.loop_indices:
        print("    Vertex: %d"
          % me.loops[loop_index].vertex_index)
```

```
print('Face list:')
for i in range(len(me.polygons)):
    print(i, ":", me.polygons[i].vertices[:])
print('Equiv_________________')
for poly in me.polygons:
    print("Polygon index: %d, length: %d"
          % (poly.index, poly.loop_total))
    for loop_index in poly.loop_indices:
        print("    Vertex: %d"
          % me.loops[loop_index].vertex_index)
#equivalent to
#   for loop_index in range(poly.loop_start,
#         poly.loop_start + poly.loop_total):
...
```

BMesh

This is a standalone module (not part of the core) which gives access to Blender's mesh-editing facilities. While Mesh is intended to be a compact representation, BMesh is intended to facilitate editing; it **does not reflect the internal state of blender**, and needs you to move information in and out as appropriate.

`https://docs.blender.org/api/current/bmesh.html` and following pages give a detailed description of the data structure of BMesh inside Blender (and cites the reference it is based on). Not all of it is exported to Python.

Remember that working with BMesh requires to follow some copy-in-copy-out liturgy.

From docs.blender.org/api/current/

Blender Python API

Search

DOCUMENTATION

Quickstart

API Overview

API Reference Usage

Best Practice

Tips and Tricks

Gotchas

Advanced

Change Log

APPLICATION MODULES

Context Access (bpy.context)

Data Access (bpy.data)

Message Bus (bpy.msgbus)

Blender 4.3 Python API Documentation

Welcome to the Python API documentation for Blender, the free and open source 3D creation suite.

This site can be used offline: Download the full documentation (zipped HTML files)

Documentation

- Quickstart: New to Blender or scripting and want to get your feet wet?
- API Overview: A more complete explanation of Python integration.
- API Reference Usage: Examples of how to use the API reference docs.
- Best Practice: Conventions to follow for writing good scripts.
- Tips and Tricks: Hints to help you while writing scripts for Blender.
- Gotchas: Some of the problems you may encounter when writing scripts.
- Advanced: Topics which may not be required for typical usage.
- Change Log: List of changes since last Blender release

APPLICATION MODULES

Context Access (bpy.context)

Data Access (bpy.data)

Message Bus (bpy.msgbus)

Operators (bpy.ops)

Types (bpy.types)

Utilities (bpy.utils)

Now for some exercises. . .